

Fondements informatiques I

Cours 2: Structures conditionnelles

Sorina Ionica `sorina.ionica@uvsq.fr`

Bilal FAYE `bilal.faye@uvsq.fr`

Les structures conditionnelles

À quoi servent-elles ?

Une structure conditionnelle permet d'exécuter certaines instructions **seulement si une condition est vérifiée**.

Exemple

Si le solde est suffisant → accepter la transaction.

Sinon → rejeter la transaction.

En Python

Une condition est une expression qui produit une valeur booléenne : True ou False.

Les trois formes que nous allons voir

if if ... else if ... elif ... else

L'instruction if

Principe

Jusqu'à présent, les instructions étaient exécutées **dans l'ordre**, les unes après les autres.

Avec `if`, certaines instructions sont exécutées **uniquement si une condition est vraie**.

Syntaxe

```
if <condition>:  
    instruction1  
    instruction2
```

Bloc d'instructions

Les instructions **indentées** sous le `if` forment un même bloc.

L'indentation permet à Python d'identifier les instructions qui appartiennent au bloc conditionnel.

L'instruction if : un exemple

Exemple

```
meteo = "pluie"  
if meteo == "pluie":  
    print("parapluie")  
    print("impermeable")  
print("Bonne journée !")
```

Que se passe-t-il ?

- `meteo == "pluie"` vérifie si `meteo` vaut "pluie".
- Cette condition renvoie `True` ou `False`.
- Les deux `print` indentés sont exécutés si la condition est vraie.
- `print("Bonne journée !")` n'est pas indenté : il est exécuté dans tous les cas.

L'instruction if-else

Principe

La structure if-else permet de choisir entre **deux blocs d'instructions** :

- le bloc if si la condition est vraie ;
- le bloc else sinon.

Syntaxe

```
if <condition>:  
    instruction(s)  
else:  
    instruction(s)
```

À retenir

- else est au même niveau d'indentation que if.
- else est suivi de deux points :.
- Un seul des deux blocs est exécuté.

L'instruction if-else : un exemple

Exemple

```
meteo = "froid"
if meteo == "pluie":
    print("parapluie")
    print("impermeable")
else:
    print("t-shirt")
    print("shorts")
print("Bonne journée !")
```

Que se passe-t-il ?

- `meteo == "pluie"` est faux : le bloc `else` est exécuté.
- Les instructions du bloc `if` ne sont donc pas exécutées.
- `print("Bonne journée !")` n'est pas indenté : il est exécuté après le `if-else`.

L'instruction elif

Principe

elif permet de tester une **nouvelle condition** lorsque la condition précédente est fausse. On peut ainsi enchaîner plusieurs conditions.

Syntaxe

```
if <condition>:
    instruction(s)
elif <condition>:
    instruction(s)
else:
    instruction(s)
```

- Le mot clé elif doit être au même niveau d'indentation que l'instruction if qu'elle complète.
- elif doit également être suivi d'une condition et de deux points (:).
- L'instruction else ne peut figurer qu'une seule fois en clôture du bloc de la condition if.

L'instruction elif : un exemple

Exemple

```
meteo = "froid"
if meteo == "pluie":
    print("parapluie")
    print("impermeable")
elif meteo == "froid":
    print("pull")
    print("echarpe")
else:
    print("t-shirt")
    print("shorts")
print("Bonne journée !")
```

- Il est possible de mettre autant de elif que l'on souhaite après une condition if.
- Les instructions elif et else sont facultatives : lorsqu'une instruction en if ou elif est définie, il n'est pas obligatoire de prévoir un else après.

Exercice : trouver l'erreur

Code

```
meteo = "froid"
if meteo == "froid":
    print("echarpe")
print("Bonne journée !")
elif meteo == "froid" or meteo == "venteux":
    print("manteau")
    print("Bonne journée !")
else:
    print("t-shirt")
print("Bonne journée !")
```

Question

Pourquoi ce code provoque-t-il une erreur ?

Les conditions et les opérateurs de comparaison

Principe

Les opérateurs de comparaison permettent de construire une condition.
Le résultat est toujours True ou False.

Opérateur	Signification
<	strictement inférieur à
>	strictement supérieur à
<=	inférieur ou égal à
>=	supérieur ou égal à
==	égal à
!=	différent de

Exemple

$a \geq b \rightarrow$ True si a est supérieur ou égal à b , False sinon.

Exemple

Vérifier si une transaction est possible

```
solde = 20.0
prix_achat_hors_taxe = 19.0
taxe = 1.08
if solde >= prix_achat_hors_taxe * taxe:
    print("Transaction possible")
else:
    print("Transaction impossible")
print("Fin")
```

À retenir

Dans la condition, la multiplication est effectuée avant la comparaison :

$$19.0 \times 1.08 = 20.52$$

Donc $20.0 \geq 20.52$ est faux.

Priorité des opérateurs

Du plus prioritaire au moins prioritaire
()
**
*, /, //, %
==, !=, <, >, <=, >=
not
and
or

Exercice

Pour $a = 3$ et $b = 2$, que valent les expressions suivantes ?

```
a, b = 3, 2
a + b * 2
a > 0 and b * 2 == 4
not a > 0 or b > 5
not (a > 0 and b > 5)
2 ** 3 // 4
17 // 5 + 17 % 5
```

Évaluation paresseuse avec and et or

Principe

Python évalue les expressions de gauche à droite et **s'arrête dès que le résultat est connu**.

x and y

Si x est False, y n'est pas évalué.

False and ...

x or y

Si x est True, y n'est pas évalué.

True or ...

Exemple

```
False and (10 / 0)    # pas d'erreur
True or (10 / 0)      # pas d'erreur
```

Exemples avec and

Premier cas

```
x = 0
if x != 0 and 2 // x == 2:
    print("Test réussi")
print("Fin")
```

Pas d'erreur : $x \neq 0$ est faux, donc la division n'est pas évaluée.

Deuxième cas

```
x = 0
if 2 // x == 2 and x != 0:
    print("Test réussi")
print("Fin")
```

Erreur : la division est évaluée en premier.

Exemples avec or

Premier cas

```
x = 0
if x == 0 or 2 // x == 2:
    print("Test réussi")
print("Fin")
```

Pas d'erreur : `x == 0` est vrai, donc la division n'est pas évaluée.

Deuxième cas

```
x = 0
if 2 // x == 2 or x == 0:
    print("Test réussi")
```

Erreur : la division est évaluée en premier.

À retenir

Avec `and` comme avec `or`, l'ordre des conditions peut changer le résultat de l'évaluation.

La portée des variables

Définition

La **portée** (*scope*) d'une variable désigne les endroits du programme où l'on peut utiliser cette variable.

Exemple

```
num1 = 1
num2 = 2
resultat = "résultat non connu"
if num1 < num2:
    resultat = "num2 est plus grand que num1"
print(resultat)    # resultat est toujours accessible
```

À retenir

En Python, un bloc `if` ne crée pas une nouvelle portée : `resultat` reste accessible après le `if`.

La portée des variables : attention

Que se passe-t-il ?

```
num1 = 2
num2 = 1
if num1 < num2:
    maximum = num2
print(maximum)
```

Erreur

La condition est fausse : le bloc if n'est pas exécuté.
maximum n'est donc jamais créée.
→ print(maximum) provoque une erreur.

Instructions if imbriquées

Principe

Un if peut être placé à l'intérieur d'un autre if. Le second if est alors exécuté seulement si le premier est vrai.

Exemple

```
num1, num2 = 2, 5
maximum = num1
if num1 < num2:
    maximum = num2
    num3 = 3
    if num2 < num3:
        maximum = num3
print(maximum)
```

À vous de réfléchir

Que vaut maximum à la fin ?

À quel endroit num3 est-elle créée ?

Qu'affiche ce programme ?

```
solde = 21.0
taxe = 1.08
prix = 19.0
nom_carte = "Daniel Dupont"
nom_client = "Daniel Dupont"
vendeur = "Fred"
vendeurs = ["Maria", "Yoann", "Emilie", "Tia"]
decouvert = True
if solde <= prix * taxe and not decouvert:
    print("Transaction refusée")
else:
    if nom_carte != nom_client:
        print("Client non valide")
    else:
        if vendeur not in vendeurs:
            print("Vendeur non autorisé")
        else:
            print("Transaction approuvée")
```

Suivez les conditions une par une pour trouver le résultat.